

Introduction Kotlin

val immutable - assignable une fois	var mutable
val p : Person?; p?.name	Null safety: Pour chaque type T, il existe T? qui prend T ou null
c?.name ?: "default"	Elvis Operator: Prendre valeur en cas de null
fun String.doSomething():String {...}	Extension functions: ajouter fonctions à classes sans sources
class Person(var i:Int) {init {...}}	init: constructeur par défaut
constructor(i:Int,j:Int):this(i) {...}	constructor: constructeurs supplémentaire
fun add(x: Int, y: Int):Int {...}	Arguments nommés possible: add(y = 1, x = 2)
class Student(...): Person(...) {...}	Héritage: appel super-constructeur. open class Person() {...}
interface Flying { val w:Wings fun fly(){...} }	class Bird: Flying { override val wings:Wings=... }
data class Person(var name:String)	Génère: getter - setters - toString()
val(name, surname) = p	Déstructuration pour data class
var p2 = p1.copy(surname="Bob")	Copie pour data class, arguments optionnels
val s = "Person[name:\${p.name},id:\${id}]"	String templates
Collections	List, MutableList, Set, MutableSet, Map, MutableMap
val l = listOf(1, 2, 3, 4)	l.any{...} l.count{...} l.max() l.filter{...} l.map{...} l.partition{...} l+l etc
Operators overload:	Array: get(i) set(i) a..b b.rangeTo(b) a in b b.contains(a)
Unary: unaryPlus() not() inc() dec()	Binary: plus(b) minus(b) times(b) div(b) mod(b) plusAssign()
val res = when(x) {1->"a" else->{"b"}}	When - Pattern Matching
p?.let {it.name="Bob" println(it)}	Scope functions - let - retourne dernière ligne du bloc
val p2 = p.apply {this.name="Bob" println(this)}	Scope functions - apply - retourne l'objet lui-même
BufferedWriter(FileWriter("a.txt")).use{ it.appendLine("Hello") }	Scope functions - use - appelé sur objets <i>Closeable</i> , ferme la variable it automatiquement en fin de bloc ou en cas d'exception
companion object {var a fun getA():A}	Companion objects contient variables/fonctions statiques

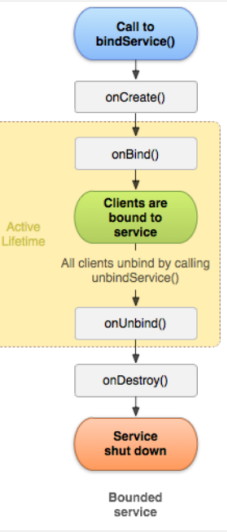
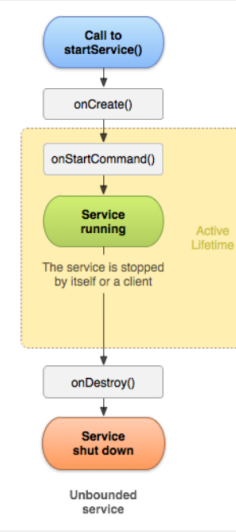
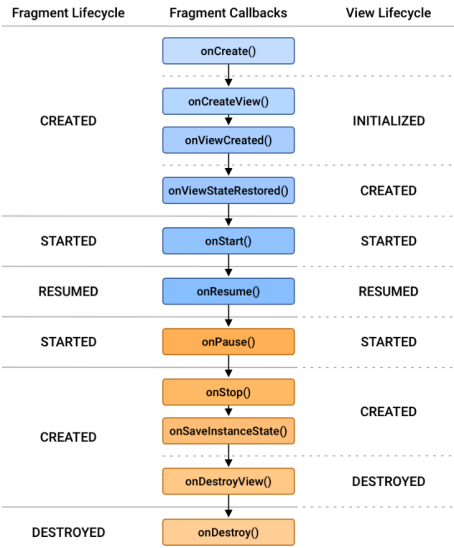
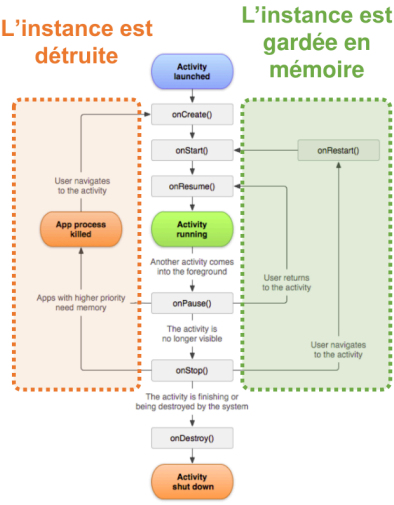
Ressources

Manifest	décrit informations essentielles pour build, OS, et store
Doit contenir	Composants d'app(Activités, Services, Broadcast receivers, Content providers), Permissions, Fonctionnalités (hard/software)
Ressources	différentes ressources et classe <i>R</i>
Contextualisation - 🚩 Lire doc pour ordre	<i>fr sw600dp night land</i> densité écran: <i>hdpi</i> taille écran: <i>normal</i>
Valeurs - string.xml	peut être regroupées en tableau
Placeholders	<string name="welcome">Hello, %1\$s!</string>
Plurals - zero one two few many	<plurals name="test"> <item quantity="one">one</item>

	<item quantity="other">more</item> </plurals>
Dimensions - dimension.xml	dp:density independent pixel, sp:scale independent pixel
Couleurs - colors.xml	-> themes.xml
Drawables	
Bitmap - png, webp, jpeg, gif	Plusieurs résolutions pour chaque type d'écran
Vector - xml	outil pour convertir depuis svg ou psd
Nine-Patch - *.9.png]	Redimensionnement contrôlé
State List	Drawable matchant différents states de la vue
Level List	Drawable matchant une valeur numérique
Layouts - LinearLayout - RelativeLayout - ConstraintLayout - ScrollView	possible de les imbriquer mais moins bonnes performances. ScrollView est vertical (mais HorizontalScrollView existe)
classe R	regroupe les ids des ressources. Accessible code ou ressources
Build - Gradle	build.gradle app et projet
Dépendances - Maven	via <i>build.gradle.kts</i> ou <i>libs.versions.toml</i> (mieux)

Activités, Fragments, Services

Activity - Stack	Doivent être déclarées dans le manifest
Cycle de vie - Inact->Stopped->Paused->Active->Paused->Stopped->Inact	Active:foreground, can not be killed Paused:visible, no focus Stopped:invisible Inactive:temporary when created/killed
État activité	activités sur stack peuvent être détruites. changement de config va recréer activité active
Sauvegarde-Restoration	onSaveInstanceState -> Bundle -> onCreate/ onRestoreInstanceState
Intents	Lance une activité sur la stack avec paramètres éventuels
Intents Explicites - activité précise	Intents Implicites - appel générique via un lien/type intent
Contracts	résultat d'une activité (moderne, non déprécié)
Fragment	plusieurs par Activity. Gérés par Fragment Manager
Service	Réaliser longues opérations en arrière-plan
Foreground - par intent ou méthode	notification visible: Lecteur Audio, Téléchargement,...
Background - par intent ou méthode	sans interface utilisateur, limité dans le temps: sync. serveur
Bounded - par méthode, bind	lié composant d'une app, détruit lorsque plus aucun n'est bind.
Broadcast Receivers	publish-subscribe, via manifest, runtime (moins de limitations)
Content Providers	accéder DB, Uri unique par donnée, CRUD
FileProvider - sous classe	partage sécurisé de fichiers entre apps à private storage
Permissions - Bonnes pratiques	Contrôle, Transparence, Minimisation
Installation	listées dans manifest, automatiquement accordées
Exécution	permissions dangereuses, listées dans manifest, pop up
Spéciales	réservées à OS ou constructeur téléphone



Interface graphique

LiveData et MVVM

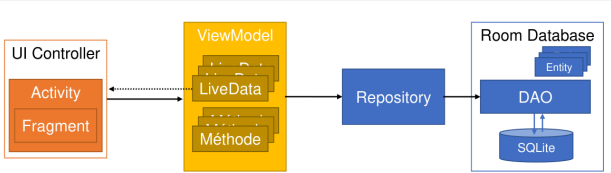
LiveData - librairie Android Jetpack	observable, lifecycle aware (ne va notifier que si actif/visible)
avantages:	update visuelle auto, pas de memory leaks
souvent créés dans un ViewModel	remplace sauvegarde état, partage données entre activité/fragment
MutableLiveData	LiveData ne sont pas modifiables
MVC - Responsabilités Activités/Fragments	gestion vues (init, update), réaction user input, API system calls, ...
MVC - Problèmes Activités/Fragments	destruction/restoration à tout moment, perte d'appels asynchrones
MVVM	Model (Room) <-> ViewModel <-> View (UI controller)

ViewModel	si associé à une activité survit aux recreations, référencé Activités/Fragments, stockage données court terme/sans persistance
particularités	n'est instancié que si utilisé, si utilisé lorsqu'activité inactive->échec
bonnes pratiques	ne pas avoir de réf vers Vue/Activité/Fragment, plusieurs par Activité, exposer LiveData uniquement et pas MutableLiveData (mais possible de cast en MutableLiveData...)

Persistance des données

dossier assets	disponible lecture seule
Fichiers privés - interne: chiffré	géré par application - mise en cache: sans garantie
Fichiers privés - externe: chemin absolu peut changer	géré par application - mise en cache
Fichiers média partagés	API <i>Mediastore</i> requêtes et résultats en Uri
SharedPreferences	key-value, géré par SDK, stockés dans XML privé interne
Base de données	Données stockées dans un fichier, SQLite
Android Room	ORM pour SQLite
KSP - Kotlin Symbol Processing API	génération de code par annotation
Repository	Single source of truth, point d'accès aux données
Entités	data class pour stocker objets, annotations
DAO	Data Access Object, interfaces pour accéder à la DB, requêtes, code généré automatiquement
Database	Définition DB, mise en relation composants, singleton
Converters	convertit types complexes afin de les stocker, par ex date
Requêtes sur LiveData	Exécuter chaque fois que table est modifiée -> <i>distinctUntilChanged</i>
Requêtes sur toutes les données	Charger en mémoire -> <i>Flow</i> ou pagination
non chiffrement des données	DB: Android 10 pour stockage interne. applicatif: SQLite+SQLCipher

Architecture



Persistance des données - Résumé

Type	Permissions	Accès autres applications	Suppression lors désinstallation
Fichiers privés: Internes	aucune	non	oui
Fichiers privés: Externes	aucune (19+)	possible	oui
Média - <i>Mediastore API</i>	READ_EXTERNAL_STORAGE	oui	non
Autres fichiers partagés (téléchargements)	aucune via <i>Storage Access Framework</i>	oui	non
Préférences	aucune	non	oui

Base de données locale	aucune	non	non
------------------------	--------	-----	-----